🧪 6.5 Hands-on Lab — Build and Deploy a Full AI + IoT Pipeline on Jetson Orin

Objective

Build a **real-time AI pipeline** on Jetson Orin that:

1. Runs a TensorRT-optimized object detection model
2. Receives IoT sensor data via MQTT
3. Fuses AI inference + sensor data for decision-making
4. Sends results to a remote monitoring dashboard

Prerequisites

- Jetson Orin with JetPack 5.x installed
- NVIDIA Container Runtime (`sudo apt install nvidia-container-toolkit`)
- Docker installed & running
- Python 3.8+ environment
- MQTT broker (e.g., Eclipse Mosquitto) running locally or in cloud
- Test camera (USB or CSI)
- At least one IoT sensor (e.g., DHT22 temperature sensor)

Step 1 — Update System and Install Dependencies

```
sudo apt update && sudo apt upgrade -y
sudo apt install python3-pip python3-opencv -y
pip3 install paho-mqtt numpy
```

Explanation:
We ensure the Jetson is fully updated, install OpenCV for video capture, and install `paho-mqtt` for MQTT communication.

Step 2 — Download a Pretrained Model from NGC

```
mkdir ~/jetson_pipeline && cd ~/jetson_pipeline
wget https://api.ngc.nvidia.com/v2/models/nvidia/tlt_peoplenet/versions/deployable_v1.0/files/resnet18_peoplenet_pru
```

Explanation:

We use NVIDIA's PeopleNet model (ETLT format) as our object detection model for testing.

Step 3 — Convert Model to TensorRT Engine

```
/opt/nvidia/tao/tao-converter \
    -k nvidia_tlt \
    -p input_1,1x3x544x960,8x3x544x960,16x3x544x960 \
    -t fp16 \
    -e peoplenet.trt \
    resnet18_peoplenet_pruned.etlt
```

Explanation:

We convert the ETLT model to a TensorRT engine optimized for FP16 inference on Jetson Orin.

Step 4 — Test TensorRT Model Inference

```
python3
```

```
import cv2, numpy as np
import tensorrt as trt

TRT_LOGGER = trt.Logger(trt.Logger.WARNING)
with open("peoplenet.trt", "rb") as f:
    runtime = trt.Runtime(TRT_LOGGER)
    engine = runtime.deserialize_cuda_engine(f.read())

print("TensorRT engine loaded successfully!")
```

Explanation:

We confirm the TensorRT engine loads without errors before building the full pipeline.

Step 5 — Set Up MQTT for Sensor Data

1. **Install Mosquitto MQTT broker locally (optional if using cloud broker):**

```
sudo apt install mosquitto mosquitto-clients -y
sudo systemctl enable mosquitto
```

2. **Publish test data from a sensor:**

```
mosquitto_pub -t "factory/temperature" -m "27.5"
```

Explanation:

We set up MQTT for IoT sensor feeds. If no physical sensor is available, publish dummy data for testing.

Step 6 — Create the AI + IoT Fusion Script

```
# fusion_pipeline.py
import cv2, time, paho.mqtt.client as mqtt

temperature_data = None

def on_message(client, userdata, msg):
    global temperature_data
    temperature_data = float(msg.payload.decode())

client = mqtt.Client()
client.connect("localhost", 1883, 60)
client.subscribe("factory/temperature")
client.on_message = on_message
client.loop_start()

cap = cv2.VideoCapture(0)  # USB camera

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # (Placeholder) – Run inference on frame here
    detected_objects = ["person", "person"]

    # Combine inference + sensor data
    if temperature_data:
        overlay = f"Detected: {len(detected_objects)} people | Temp: {temperature_data} C"
        cv2.putText(frame, overlay, (20, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)

    cv2.imshow("AI + IoT Pipeline", frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Explanation:

We read live camera frames, perform AI inference (placeholder here), and overlay temperature sensor data on the video feed.

Step 7 — Send Results to Remote Monitoring Dashboard

- Push fused results to **Grafana** or **AWS IoT Core** for visualization.
  Example using MQTT:

```
mosquitto_pub -t "factory/results" -m "2 people detected, Temp=27.5C"
```

Explanation:

This allows central monitoring of edge device outputs.

Step 8 — Enable Jetson Performance Mode

```
sudo nvpmodel -m 0
sudo jetson_clocks
```

Explanation:

This sets Jetson Orin to max performance mode for benchmarking.

Step 9 — Test and Validate

- Run `python3 fusion_pipeline.py`
- Verify AI inference and sensor data appear together.
- Check MQTT logs for results being published.
- Simulate different sensor readings to see AI + IoT decision changes.

Step 10 — Optional: Containerize the Pipeline

```
docker build -t jetson-pipeline .
docker run --runtime nvidia --network host jetson-pipeline
```

Explanation:

Containerizing makes it easy to deploy updates via OTA without touching the OS.
